

NCERT Physics Hybrid RAG Project Report

Overview

This report documents the design, architecture, working flow, frontend-backend integration, and documentation structure of the NCERT Physics Hybrid RAG system. The application combines a Streamlit interface with a FastAPI backend and uses a health check plus a query endpoint to support grounded textbook question answering [cite:564][cite:548].

The purpose of the system is to answer questions from NCERT Physics while keeping the answer tied to retrieved textbook evidence. This grounded-answer pattern is aligned with Retrieval-Augmented Generation design, where retrieval improves factual support and traceability before generation [cite:564][cite:587].

Project objective

The project aims to create a practical educational question-answering system over NCERT Physics content. The design emphasizes three outcomes: reliable backend connectivity, transparent retrieval evidence, and a usable web interface for live questioning and demonstration [cite:548][cite:566].

A secondary objective is presentation quality. In project evaluations, it is not enough for a model to answer a question; the system must also show that the backend is running, the frontend is connected, and the response is grounded in source material [cite:549][cite:583].

System components

Frontend

The frontend is built in Streamlit and uses chat-oriented components to create a simple question-answering interface. Streamlit provides `st.chat_input` for accepting user prompts, `st.chat_message` for rendering the conversation, and `st.session_state` for preserving messages across reruns [cite:566][cite:573].

In this project, the Streamlit app first checks whether the backend is available by calling the health endpoint. After that, user questions are sent to the query endpoint, and the frontend displays the response as assistant content [cite:564][cite:548].

Backend

The backend is implemented using FastAPI, which is commonly used to build performant Python APIs with clear routing and production-oriented capabilities. FastAPI also supports middleware patterns such as CORS configuration when frontend and backend are split across different origins [cite:493][cite:587].

The backend exposes at least two important endpoints in this workflow: `/health` and `/query/ask`. The health route is used for availability checks, while the query route handles user questions and returns the answer payload [cite:548][cite:549].

Retrieval layer

The project follows a hybrid retrieval design, combining multiple retrieval approaches rather than depending on only one method. This structure improves robustness because semantic retrieval, keyword retrieval, and graph-based retrieval can each capture different types of relevance for textbook questions.

The fusion layer combines the outputs of these retrieval methods and selects the most useful chunks for answer generation. This is the stage that connects retrieval quality to final answer quality, because the generation stage can only be as reliable as the evidence it receives.

Source knowledge

The core knowledge source is the NCERT Physics textbook, typically processed into chunks and indexed for retrieval. In a hybrid RAG workflow, this content may be used to support semantic indexing, keyword lookup, and graph relationships between concepts.

Application flow

The application flow starts when the user enters a question in the Streamlit interface. The frontend then communicates with the backend, which processes the query and returns a grounded answer [cite:564][cite:571].

A simplified request flow is:

1. User opens the Streamlit app.
2. Frontend checks GET `/health`.
3. User enters a question.
4. Frontend sends GET `/query/ask` with query parameters.
5. Backend validates the request.
6. Retrieval modules search for relevant chunks.
7. Fusion selects the best evidence.
8. The answer generation stage produces a response tied to retrieved content.
9. The API returns JSON to the frontend.
10. Streamlit displays the answer and any citations.

This flow is especially suitable for demos because it separates system-health verification from actual question answering. Health checks are a recognized pattern for observability and service readiness in modern backend systems [cite:549][cite:548].

Frontend-backend integration

The frontend and backend are connected through Python HTTP requests rather than browser-side JavaScript in the current Streamlit design. This makes the integration straightforward because the Streamlit server itself performs the API calls to FastAPI [cite:564][cite:571].

The health-check logic confirms whether the backend is responding. Once that is successful, the query request can be sent to `/query/ask`, and the returned JSON is parsed and rendered in the chat interface [cite:548][cite:564].

The use of session state is important here because Streamlit reruns the script after user interactions. `st.session_state` preserves prior messages so that the chat looks continuous instead of resetting after each request [cite:573].

Reliability considerations

Health checks

A health endpoint improves the user experience because the app can report whether the backend is reachable before the user tries to ask a question. This reduces confusion and makes debugging easier when the backend is not running or when a deployment issue occurs [cite:549][cite:548].

CORS and split deployment

If the project later moves from Streamlit-based server-side requests to a browser-based frontend that calls the API directly, CORS configuration becomes important. FastAPI documents CORS support through middleware so that approved frontend origins can access the API safely when they are hosted separately [cite:493].

Error handling

A production-quality frontend should handle backend failure gracefully by showing user-friendly error messages, timeout warnings, and empty states. This is especially important in educational demos where the evaluator may test failure cases or unsupported questions.

Documentation strategy

A strong project should include a root README, clear setup instructions, usage examples, and architecture documentation because the README is the first entry point for users and evaluators [cite:583][cite:580]. The README should explain what the system does, how to run it, which files matter, and how the components connect [cite:586][cite:583].

Mermaid diagrams are useful for architecture documentation because they let the system design stay in plain text, version control, and Markdown. Mermaid supports architecture and flowchart-oriented visualization that can be pasted into the Mermaid Live Editor for quick viewing and refinement [cite:585][cite:588].

Architecture explanation

The architecture has five practical layers:

Layer	Role	Notes
User layer	Accepts questions	Streamlit chat interface [cite:566]
Client layer	Sends API requests	Streamlit using Python requests [cite:564]
API layer	Validates and routes requests	FastAPI endpoints [cite:548][cite:493]
Retrieval layer	Searches source content	Semantic, keyword, and graph retrieval
Answer layer	Produces grounded response	Returns answer plus citations

This layered structure is easier to explain in evaluations because each step has a clear responsibility. The frontend handles interaction, the backend handles orchestration, the retrievers find evidence, and the answer layer formats the output.

Mermaid architecture code

The architecture can be represented in Mermaid as follows:

```
flowchart TD
    U[User]
    FE[Streamlit Frontend]
    HC[Health Endpoint]
    QA[Query API]
    SR[Semantic Retrieval]
    KR[Keyword Retrieval]
    GR[Graph Retrieval]
    HF[Hybrid Fusion]
    EV[Relevant Evidence Chunks]
    AG[Grounded Answer Generation]
    JS[JSON Response]
    KB[NCERT Physics Knowledge Base]

    U --> FE
    FE --> HC
    FE --> QA
    QA --> SR
    QA --> KR
    QA --> GR
    KB --> SR
    KB --> KR
    KB --> GR
    SR --> HF
    KR --> HF
    GR --> HF
    HF --> EV
    EV --> AG
    AG --> JS
    JS --> FE
    FE --> U
```

This diagram is intentionally simple so that it can be explained quickly in a viva, presentation, or live review. Mermaid flowcharts are especially useful in this setting because they are text-based and easy to modify without redrawing from scratch [cite:585][cite:588].

Evaluation strengths

The project already has several strengths:

- Clear separation between frontend and backend responsibilities.
- A visible backend health check for operational confidence [cite:549][cite:548].
- A chat-based interface that is easy for evaluators to use [cite:566].
- A hybrid retrieval framing that is stronger than single-method retrieval for many educational QA scenarios.
- An architecture that can be documented and demonstrated clearly with Mermaid and Markdown [cite:585][cite:583].

Suggested next improvements

To make the project even stronger, the following additions would help:

1. Add a frontend sidebar for `top_k`, source selection, or model settings.
2. Show retrieval evidence explicitly in the UI using expanders or tabs.
3. Include logging for query duration and retrieval scores.
4. Add deployment instructions for a public demo environment.
5. Add screenshots and sample questions to the README.
6. Add a limitations section explaining unsupported questions and grounding boundaries.

Conclusion

The NCERT Physics Hybrid RAG project is a well-structured educational QA system built around a Streamlit frontend and FastAPI backend. Its architecture supports a clear flow from user question to grounded answer, and its documentation can be effectively communicated through a README, Mermaid architecture diagram, and a detailed project report [cite:583][cite:585][cite:564].